

Technical Interview Planning Document

Mock Interview Recording:

<https://app.clipchamp.com/consumer/editor?driveId=26403B6B7F0096AE&folderId=26403B6B7F0096AE!saecf31777fb546eeb37db4af1c63031e&itemId=26403B6B7F0096AE!sebca69d374b14921ada09150ffbd070f>

Find the Middle of a Chain: Given a chain of items, return the item in the middle of the chain. If the chain has an even number of items, return the second of the two middle items. You may not use built-in list or string manipulation methods (e.g., `.reverse()`, `.remove()`, slicing, etc.). You must implement your own solution using nodes, pointers, and the custom data structures taught in this course.

Input: [1, 2, 3, 4, 5] Output: 3

Input: [1, 2, 3, 4, 5, 6] Output: 4

<https://leetcode.com/playground/AWfodEow>

Problem Restatement and Key Constraints

Problem: You are given a chain of items, from the chain provided you need to return the item in the middle of the chain. If there is an even number of items you are to select the second of the two middle items. You may not use built-in list or string manipulation methods, and instead you must make your own solution using nodes, pointers, and custom data structures that were taught in this course.

Key Constraints:

- You cannot use any built in list or string manipulation methods
- You must use your own node pointers and custom data structures

Clarifying Questions to Ask:

- This is a list including only integers?
- In the example the numbers being used don't really matter, they're simply there to help the user understand which number in the chain is being returned?
- Should I expect the format to always be in a list like format or can it be a string separated by only spaces?

Inputs and Expected Outputs

Example Input 1:

None

`find_middle([1, 2, 3, 4, 5])`**Expected Output: 3****Example Input 2**

None

`find_middle([1, 2, 3, 4, 5, 6])`**Expected Output: 4****Example Input 3 (Edge Case):**

None

`find_middle([])`**Expected Output: 0** (not a negative one because it is solvable, it just needs zero coins)

Approach 1: [Brute Force or Initial Approach]

Algorithm Description: This algorithm finds the middle element of a chain (linked list) by first determining the total number of nodes, then locating the middle position based on that length. It performs one traversal to count the nodes, calculates the middle index (choosing the second middle in the case of an even number of nodes), and then performs a second traversal to reach and return the middle node's value.

Pseudocode:

None

`FUNCTION find_middle(head):` `SET length TO 0` `SET current TO head`

```

// FIRST PASS: count nodes
WHILE current is not null:
    length = length + 1
    current = current.next

// compute middle index
SET middle_index TO length / 2 (integer division)

// SECOND PASS: move to middle
SET current TO head
SET i TO 0

WHILE i < middle_index:
    current = current.next
    i = i + 1

RETURN current.data

```

Time Complexity: $O(n)$, it's still near linear but it passes through the list twice

Space Complexity: $O(1)$

Why Start with This Approach: You start with a brute force solution because it's simpler to understand and ensures you get a correct working answer by clearly breaking the problem into steps. Once that works, you can then improve it with a more efficient approach like the slow/fast pointer method.

Approach 2: [Optimized Approach] (if applicable)

Algorithm Description: This algorithm finds the middle element of a chain (linked list) using two pointers that traverse the list at different speeds. A "slow" pointer moves one node at a time while a "fast" pointer moves two nodes at a time; when the fast pointer reaches the end of the list, the slow pointer will be in the middle. This approach returns the second middle element automatically when the list has an even number of nodes and completes the task in a single traversal.

Pseudocode:

None

FUNCTION build_chain(input_string):

 SET head TO null

 SET tail TO null

 SET num TO 0

 SET reading_number TO false

 FOR each character ch in input_string:

 IF ch is a digit:

 num = num * 10 + value of ch

 reading_number = true

 ELSE:

 IF reading_number is true:

 CREATE new_node with data = num

 IF head is null:

 head = new_node

 tail = new_node

 ELSE:

 tail.next = new_node

 tail = new_node

 num = 0

 reading_number = false

 IF reading_number is true:

 CREATE new_node with data = num

 IF head is null:

 head = new_node

 ELSE:

 tail.next = new_node

```
RETURN head
```

```
FUNCTION find_middle(head):
```

```
    SET slow TO head
```

```
    SET fast TO head
```

```
    WHILE fast is not null AND fast.next is not null:
```

```
        slow = slow.next
```

```
        fast = fast.next.next
```

```
    RETURN slow.data
```

```
INPUT string like "[1, 2, 3, 4, 5]"
```

```
head = build_chain(input)
```

```
middle_value = find_middle(head)
```

```
OUTPUT middle_value
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Why This is Better: This algorithm finds the middle element of a chain (linked list) using two pointers that traverse the list at different speeds. A "slow" pointer moves one node at a time while a "fast" pointer moves two nodes at a time; when the fast pointer reaches the end of the list, the slow pointer will be in the middle. This approach returns the second middle element automatically when the list has an even number of nodes and completes the task in a single traversal.

Key Communication Points

What to Emphasize During Explanation:

1. Problem Clarification Phase:

- Confirm the input is a singly linked list (not an array) and that we're returning the middle node's value
- Clarify the rule for even-length lists: return the second middle element
- Note constraints like not using built-in list operations or converting to arrays

2. When Discussing Your Approach:

- Start with a simple brute force idea: count nodes first, then traverse again to the middle
- Then introduce the optimized approach using slow and fast pointers for a single-pass solution
- Explain why the two-pointer method is a common pattern for linked list problems

3. When Explaining Your Code:

- Walk through how the slow pointer moves 1 step and fast pointer moves 2 steps each loop
- Emphasize the loop condition that prevents null pointer errors (`fast != null && fast.next != null`)
- Highlight that when fast reaches the end, slow is automatically at the middle

4. Complexity Trade-offs:

- Brute force: $O(n)$ time but requires two passes through the list
- Optimized approach: $O(n)$ time with only one pass, improving efficiency
- Both use $O(1)$ extra space, but the optimized version is preferred for cleaner traversal and scalability

Example Walkthrough Plan

Use this example during the interview:

None

```
find_middle([1, 2, 3, 4, 5])
```

Step-by-step walkthrough for [1, 2, 3, 4, 5] (optimized slow/fast pointer)

- Start with a linked list: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow \text{null}$
 - Initialize both pointers at the head:
 - `slow = 1`
 - `fast = 1`
-

Iteration 1

- Move pointers:
 - slow moves 1 step → 2
 - fast moves 2 steps → 3
 - Current state:
 - slow = 2, fast = 3
-

Iteration 2

- Move pointers again:
 - slow moves → 3
 - fast moves → 5
 - Current state:
 - slow = 3, fast = 5
-

Stop condition

- Check loop condition:
 - fast.next is null, so the loop stops
-

Final result

- slow is pointing at 3
- Return value: **3 (middle of the list)**

During walkthrough, emphasize:

- Both pointers start at the same position, so traversal begins in sync
 - Fast pointer is moving twice as fast, starting to "pull ahead" to reach the end quicker
 - Slow pointer is now exactly in the middle of the list as fast approaches the end
 - Fast pointer reaching the end signals that slow is guaranteed to be at the middle
 - This works in a single pass and automatically returns the correct middle (second middle for even-length lists)
-

Python Implementation

Python

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def build_chain_from_list_string(s):
    head = None
    tail = None

    num = 0
    reading_number = False

    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
            reading_number = True
        else:
            if reading_number:
                new_node = Node(num)

                if head is None:
                    head = new_node
                    tail = new_node
                else:
                    tail.next = new_node
                    tail = new_node

            num = 0
            reading_number = False

    # handle last number
    if reading_number:
        new_node = Node(num)
        if head is None:
            head = new_node
        else:
            tail.next = new_node
```



```

return head

def find_middle(head):
    slow = head
    fast = head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

    return slow.data

# ---- Test ----
s = "[1, 2, 3, 4, 5]"
head = build_chain_from_list_string(s)
print(find_middle(head)) # Output: 3

s2 = "[1, 2, 3, 4, 5, 6]"
head2 = build_chain_from_list_string(s2)
print(find_middle(head2)) # Output: 4

```

Key implementation points to mention:

- Use two pointers (slow and fast) initialized at the head of the linked list to traverse the structure
- Move slow by 1 node and fast by 2 nodes each iteration to efficiently reach the middle in a single pass
- Use the loop condition while fast != null and fast.next != null to safely avoid null pointer errors and correctly stop at the end of the list
- When the loop ends, slow will be positioned at the middle node (automatically the second middle in even-length lists)
- The algorithm runs in $O(n)$ time with $O(1)$ extra space, making it more efficient than a two-pass (brute force) solution